Part 2 – Dynamic Programming: Autonomous Drone Rescue

BITS Pilani WILP – Deep Reinforcement Learning | Lab Assignment 1

Group Number: 32

Configuration for G=32 (ends in 2, even digit):

- Grid size: **5×5**
- Max battery: **10 units**
- Wind probability: **20%**
- Grid objects: 2 rescue targets, 1 charging station, 3 danger zones, 2 blocked cells

In [2]:
```
import datetime, platform, socket, os

print("=" * 60)
print("  EXECUTION METADATA")
print("=" * 60)
print(f"Timestamp         : {datetime.datetime.now().strftime('%Y-%m-%d %H:%
print(f"Hostname          : {socket.gethostname()}")
print(f"Platform          : {platform.platform()}")
print(f"Python Version    : {platform.python_version()}")
try:
    with open("/etc/machine-id") as f:
        vm_id = f.read().strip()
except Exception:
    vm_id = "N/A (not a Linux VM or file missing)"
print(f"Virtual Machine ID: {vm_id}")
print(f"Working Directory : {os.getcwd()}")
print("=" * 60)
```

```
============================================================
  EXECUTION METADATA
============================================================
Timestamp         : 2026-06-07 14:40:23
Hostname          : 2025aa05383
Platform          : Linux-5.14.0-503.14.1.el9_5.x86_64-x86_64-with-glibc2.34
Python Version    : 3.12.7
Virtual Machine ID: ec2dea449c6695d5764ba96a11047ce4
Working Directory : /home/cloud/Downloads
============================================================
```

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.colors as mcolors
from itertools import product
```

```
import time, warnings
warnings.filterwarnings("ignore")

G = 32
print(f"Group Number: G = {G}")
print(f"Last digit  : {G % 10}  → 5×5 grid, max battery = 10, wind prob = 20
```

```
Group Number: G = 32
Last digit  : 2  → 5×5 grid, max battery = 10, wind prob = 20%
```

Grid Configuration

The 5×5 grid is laid out as follows (row 0 = top, col 0 = left):

Symbol	Meaning
S	Start position (fixed at top-left: row 0, col 0)
F	Free / Safe cell
D	Dangerous zone
R	Rescue target
C	Charging station
W	Wind zone
X	Blocked cell

Starting battery: 10 units (even digit → max = 10)

Placement rules (ends 0-4): 2R, 1C, 3D, 2X, remaining cells are F or W.

In [5]:
```
# ──────────────────────────────────────────────────────────────
# Grid Configuration for Group 32 (5×5 grid)
# Positions are (row, col) tuples; row 0 = top, col 0 = left.
# Start (S) is fixed at top-left corner (0,0) per assignment rules.
# ──────────────────────────────────────────────────────────────

GRID_ROWS = 5
GRID_COLS = 5
MAX_BATTERY = 10       # even last digit
WIND_PROB   = 0.20     # ends in 0-4 → 20%
MAX_STEPS   = 50       # 5×5 grid limit

# Static cell type map (before any rescues)
# S=start, F=free, D=danger, R=rescue, C=charging, W=wind, X=blocked
GRID_MAP = [
    ["S", "F", "D", "R", "F"],   # row 0
    ["F", "W", "F", "D", "X"],   # row 1
    ["R", "F", "C", "F", "F"],   # row 2
    ["D", "X", "F", "W", "F"],   # row 3
    ["F", "F", "F", "F", "F"],   # row 4
]
```

```python
# Derived position sets for quick lookup
START_POS       = (0, 0)
RESCUE_TARGETS  = [(0, 3), (2, 0)]          # 2 rescue targets
CHARGING_CELLS  = [(2, 2)]                   # 1 charging station
DANGER_CELLS    = [(0, 2), (1, 3), (3, 0)]  # 3 danger zones
BLOCKED_CELLS   = [(1, 4), (3, 1)]          # 2 blocked cells
WIND_CELLS      = [(1, 1), (3, 3)]          # wind zones

print("Grid configuration (row, col):")
print(f"  Start position  : {START_POS}")
print(f"  Rescue targets  : {RESCUE_TARGETS}")
print(f"  Charging station: {CHARGING_CELLS}")
print(f"  Danger zones    : {DANGER_CELLS}")
print(f"  Blocked cells   : {BLOCKED_CELLS}")
print(f"  Wind zones      : {WIND_CELLS}")
print(f"\nMax battery : {MAX_BATTERY}")
print(f"Wind prob   : {WIND_PROB}")
print(f"Max steps   : {MAX_STEPS}")

# ── Render the initial grid ──────────────────────────────────────────
print("\nInitial Grid Layout:")
print("     " + "  ".join([f"C{c}" for c in range(GRID_COLS)]))
for r in range(GRID_ROWS):
    row_str = "  ".join([f"{GRID_MAP[r][c]:2s}" for c in range(GRID_COLS)])
    print(f"R{r} [ {row_str} ]")
```

```
Grid configuration (row, col):
  Start position  : (0, 0)
  Rescue targets  : [(0, 3), (2, 0)]
  Charging station: [(2, 2)]
  Danger zones    : [(0, 2), (1, 3), (3, 0)]
  Blocked cells   : [(1, 4), (3, 1)]
  Wind zones      : [(1, 1), (3, 3)]

Max battery : 10
Wind prob   : 0.2
Max steps   : 50

Initial Grid Layout:
     C0  C1  C2  C3  C4
R0 [ S   F   D   R   F  ]
R1 [ F   W   F   D   X  ]
R2 [ R   F   C   F   F  ]
R3 [ D   X   F   W   F  ]
R4 [ F   F   F   F   F  ]
```

Expected Outcome 1 – Custom Drone Rescue Environment (1 Mark)

State Representation

Each state is a tuple: **(row, col, battery, rescue_status)**

- `(row, col)` – drone position on the 5×5 grid → 25 possibilities

- `battery` – current battery level from 0 to MAX_BATTERY → 11 values
- `rescue_status` – bitmask (2 bits for 2 rescue targets) → 4 combinations (00=none, 01=first rescued, 10=second rescued, 11=both)

Total theoretical states = 25 × 11 × 4 = **1100 states**

In [7]:
```
# ──────────────────────────────────────────────────────────────────────
# DroneRescueEnv: Custom MDP environment for the autonomous rescue drone.
#
# State  : (row, col, battery, rescue_mask)
#            rescue_mask is an integer bitmask over RESCUE_TARGETS list.
#            bit i = 1 means rescue target i has already been rescued.
#
# Actions: 0=Up, 1=Down, 2=Left, 3=Right, 4=Hover
# ──────────────────────────────────────────────────────────────────────

# Action constants
UP, DOWN, LEFT, RIGHT, HOVER = 0, 1, 2, 3, 4
ACTION_NAMES = {UP: "↑ Up", DOWN: "↓ Down", LEFT: "← Left",
                RIGHT: "→ Right", HOVER: "● Hover"}
ACTION_DELTAS = {UP: (-1, 0), DOWN: (1, 0), LEFT: (0, -1), RIGHT: (0, 1)}

# Reward constants (as specified in assignment)
REWARD_RESCUE    =  20
REWARD_DANGER    = -10
REWARD_BATTERY_0 = -20
REWARD_CHARGE    =   5
REWARD_MOVE      =  -1

NUM_RESCUES = len(RESCUE_TARGETS)   # 2

class DroneRescueEnv:
    """
    Custom Finite MDP environment representing the 5×5 drone rescue grid.

    Methods
    -------
    reset()       : Resets environment to the initial state and returns it.
    step(action)  : Executes one action, returns (next_state, reward, done,
    render()      : Prints the current grid state to stdout.
    transitions() : Returns the full probability transition table T(s,a,s')
    """

    def __init__(self):
        """Initialise environment constants and call reset()."""
        self.grid_rows    = GRID_ROWS
        self.grid_cols    = GRID_COLS
        self.max_battery  = MAX_BATTERY
        self.wind_prob    = WIND_PROB
        self.max_steps    = MAX_STEPS
        self.n_rescues    = NUM_RESCUES
        self.reset()

    # ── reset ──────────────────────────────────────────────────────────
    def reset(self):
```

```
        """
        Reset the environment to the initial state.
        Drone starts at START_POS with full battery and no rescues done.
        Returns the initial state tuple.
        """
        self.pos          = START_POS          # (row, col)
        self.battery      = self.max_battery   # start fully charged
        self.rescue_mask  = 0                  # no rescues done yet
        self.steps        = 0
        self.done         = False
        return self._get_state()

    def _get_state(self):
        """Return the current state as a tuple (row, col, battery, rescue_ma
        return (self.pos[0], self.pos[1], self.battery, self.rescue_mask)

    def _cell_type(self, r, c, rescue_mask=None):
        """
        Return the cell type at (r, c) given the current rescue_mask.
        Rescued targets become free cells (F).
        """
        if rescue_mask is None:
            rescue_mask = self.rescue_mask
        base = GRID_MAP[r][c]
        if base == "R":
            # Check if this rescue target has been picked up
            idx = RESCUE_TARGETS.index((r, c))
            if rescue_mask & (1 << idx):
                return "F"   # already rescued → free cell
        return base

    def _is_blocked(self, r, c):
        """Return True if (r,c) is out-of-bounds or a blocked cell."""
        if r < 0 or r >= self.grid_rows or c < 0 or c >= self.grid_cols:
            return True
        return GRID_MAP[r][c] == "X"

    def valid_actions(self, state=None):
        """
        Return list of valid action indices for the given state (or current
        All 5 actions are always considered; movement into blocked/OOB cells
        handled by keeping the drone in place (still costs 1 battery).
        """
        return [UP, DOWN, LEFT, RIGHT, HOVER]

    # ── step ──────────────────────────────────────────────────────────────
    def step(self, action):
        """
        Execute one action and advance the environment by one step.

        Parameters
        ----------
        action : int — one of UP/DOWN/LEFT/RIGHT/HOVER

        Returns
        -------
```

```
        next_state : tuple  — (row, col, battery, rescue_mask)
        reward     : float  — immediate reward
        done       : bool   — True if episode terminated
        info       : dict   — extra diagnostic info
        """
        if self.done:
            raise RuntimeError("Episode is done. Call reset() first.")

        r, c = self.pos
        reward = 0
        info   = {}

        # ── Compute intended movement direction ──────────────────────────
        if action == HOVER:
            dr, dc = 0, 0
        else:
            dr, dc = ACTION_DELTAS[action]

        # ── Wind disturbance (only when standing on a wind cell) ─────────
        if GRID_MAP[r][c] == "W" and action != HOVER:
            if np.random.rand() < self.wind_prob:
                # Wind redirects movement uniformly to one of 4 directions
                wind_action = np.random.choice([UP, DOWN, LEFT, RIGHT])
                dr, dc = ACTION_DELTAS[wind_action]
                info["wind_redirected"] = True

        # ── Compute candidate new position ───────────────────────────────
        nr, nc = r + dr, c + dc

        # ── Blocked / OOB → stay in place ────────────────────────────────
        if self._is_blocked(nr, nc):
            nr, nc = r, c
            info["blocked"] = True

        self.pos = (nr, nc)

        # ── Battery update ───────────────────────────────────────────────
        if action == HOVER and GRID_MAP[nr][nc] == "C":
            # Hovering on charging station charges +2 (capped at max)
            self.battery = min(self.battery + 2, self.max_battery)
            reward += REWARD_CHARGE
        else:
            self.battery -= 1   # every action costs 1 battery

        # ── Cell-entry events ────────────────────────────────────────────
        cell = self._cell_type(nr, nc)

        if cell == "R":
            # Rescue a civilian: get reward, mark target as rescued
            idx = RESCUE_TARGETS.index((nr, nc))
            self.rescue_mask |= (1 << idx)
            reward += REWARD_RESCUE
            info["rescued"] = idx

        elif cell == "C" and action != HOVER:
            # Entering (not hovering on) a charging station: full recharge
```

```
            self.battery = self.max_battery
            reward += REWARD_CHARGE

        elif cell == "D":
            # Danger zone: heavy penalty but episode continues
            reward += REWARD_DANGER

        else:
            # Normal movement cost
            reward += REWARD_MOVE

        # ── Check termination conditions ──────────────────────────────
        self.steps += 1

        if self.battery <= 0:
            reward += REWARD_BATTERY_0
            self.done = True
            info["termination"] = "battery_exhausted"

        elif bin(self.rescue_mask).count("1") == self.n_rescues:
            self.done = True
            info["termination"] = "all_rescued"

        elif self.steps >= self.max_steps:
            self.done = True
            info["termination"] = "max_steps_exceeded"

        return self._get_state(), reward, self.done, info

    # ── render ────────────────────────────────────────────────────────
    def render(self, policy=None, state=None):
        """
        Print the current grid with the drone's position marked as '@'.
        Optionally overlay a policy arrow for the given state.
        """
        r, c = self.pos if state is None else (state[0], state[1])
        mask = self.rescue_mask if state is None else (state[3] if state els

        print(f"\nBattery: {self.battery}/{self.max_battery} | "
              f"Rescues: {bin(mask).count('1')}/{self.n_rescues} | "
              f"Steps: {self.steps}/{self.max_steps}")
        print("     " + "  ".join([f"C{col}" for col in range(self.grid_cols
        for row in range(self.grid_rows):
            row_str = ""
            for col in range(self.grid_cols):
                if row == r and col == c:
                    cell = " @"
                else:
                    ct = self._cell_type(row, col, mask)
                    cell = f" {ct}"
                row_str += cell + " "
            print(f"R{row} [{row_str}]")


# ── Quick sanity test ─────────────────────────────────────────────────
env = DroneRescueEnv()
```

```
print("Initial state:", env.reset())
env.render()
print("\nTaking a few test steps:")
for a_name, a in [("Right", RIGHT), ("Down", DOWN), ("Right", RIGHT)]:
    s, r, done, info = env.step(a)
    print(f"  Action={a_name:5s} → state={s}, reward={r:+.0f}, done={done},
```

```
Initial state: (0, 0, 10, 0)

Battery: 10/10 | Rescues: 0/2 | Steps: 0/50
     C0  C1  C2  C3  C4
R0 [ @   F   D   R   F ]
R1 [ F   W   F   D   X ]
R2 [ R   F   C   F   F ]
R3 [ D   X   F   W   F ]
R4 [ F   F   F   F   F ]

Taking a few test steps:
  Action=Right → state=(0, 1, 9, 0), reward=-1, done=False, {}
  Action=Down  → state=(1, 1, 8, 0), reward=-1, done=False, {}
  Action=Right → state=(1, 2, 7, 0), reward=-1, done=False, {}
```

Expected Outcome 2 – Dynamic Programming Solution (2 Marks)

State Space Enumeration

We enumerate all reachable states and build the full transition model T(s, a) → [(prob, s', reward)]. Value Iteration is then applied with threshold $\theta = 10^{-3}$.

In [9]:
```
# ──────────────────────────────────────────────────────────────────────
# State Space Enumeration
# State = (row, col, battery, rescue_mask)
# ──────────────────────────────────────────────────────────────────────

def enumerate_states():
    """
    Generate all theoretically valid states (row, col, battery, rescue_mask)
    Excludes blocked cells and battery=0 (terminal) since no actions take pl

    Returns
    -------
    states : list of tuples — all valid non-terminal states
    """
    states = []
    for r in range(GRID_ROWS):
        for c in range(GRID_COLS):
            if GRID_MAP[r][c] == "X":   # skip blocked cells
                continue
            for bat in range(1, MAX_BATTERY + 1):   # 1..MAX_BATTERY
                for mask in range(1 << NUM_RESCUES): # 0..2^NUM_RESCUES - 1
                    states.append((r, c, bat, mask))
    return states
```

```
def is_terminal(state):
    """
    Return True if this state represents an episode-ending condition:
    battery=0 or all rescue targets rescued.
    """
    r, c, bat, mask = state
    if bat <= 0:
        return True
    if bin(mask).count("1") == NUM_RESCUES:
        return True
    return False


def compute_transitions(states):
    """
    Build the transition table for all (state, action) pairs.

    Returns
    -------
    T : dict
        T[(state, action)] = list of (probability, next_state, reward)
        where probabilities sum to 1.0.
    """
    state_set = set(states)
    T = {}

    for state in states:
        if is_terminal(state):
            continue  # no transitions from terminal states
        r, c, bat, mask = state

        for action in [UP, DOWN, LEFT, RIGHT, HOVER]:
            transitions = []

            # Determine base movement deltas
            if action == HOVER:
                move_list = [(1.0, 0, 0)]   # (prob, dr, dc) — no movement
            else:
                dr0, dc0 = ACTION_DELTAS[action]
                if GRID_MAP[r][c] == "W":
                    # Wind cell: with WIND_PROB the direction is randomised
                    # Intended direction with probability (1 - WIND_PROB)
                    move_list = [(1 - WIND_PROB, dr0, dc0)]
                    for wind_a in [UP, DOWN, LEFT, RIGHT]:
                        dwr, dwc = ACTION_DELTAS[wind_a]
                        move_list.append((WIND_PROB / 4, dwr, dwc))
                else:
                    move_list = [(1.0, dr0, dc0)]

            # Compute outcomes for each possible movement
            for prob, dr, dc in move_list:
                if prob == 0:
                    continue

                nr, nc = r + dr, c + dc
```

```
        # Blocked / OOB → stay in place
        if nr < 0 or nr >= GRID_ROWS or nc < 0 or nc >= GRID_COLS:
            nr, nc = r, c
        elif GRID_MAP[nr][nc] == "X":
            nr, nc = r, c

        # Battery update
        new_bat = bat
        reward  = 0.0

        if action == HOVER and GRID_MAP[nr][nc] == "C":
            new_bat = min(bat + 2, MAX_BATTERY)
            reward  = REWARD_CHARGE
        else:
            new_bat = bat - 1

        # Cell-entry events
        new_mask = mask
        base_cell = GRID_MAP[nr][nc]

        if base_cell == "R":
            idx = RESCUE_TARGETS.index((nr, nc))
            if not (mask & (1 << idx)):  # not yet rescued
                new_mask = mask | (1 << idx)
                reward  += REWARD_RESCUE
            else:
                reward += REWARD_MOVE     # already rescued → free c
        elif base_cell == "C" and action != HOVER:
            new_bat  = MAX_BATTERY        # full recharge on entry
            reward  += REWARD_CHARGE
        elif base_cell == "D":
            reward  += REWARD_DANGER
        else:
            reward  += REWARD_MOVE

        # Battery exhausted penalty
        if new_bat <= 0:
            reward += REWARD_BATTERY_0
            next_state = (nr, nc, 0, new_mask)
        else:
            next_state = (nr, nc, new_bat, new_mask)

        transitions.append((prob, next_state, reward))

    # Merge duplicate next_states
    merged = {}
    for prob, ns, rw in transitions:
        if ns not in merged:
            merged[ns] = [0.0, 0.0]
        merged[ns][0] += prob
        merged[ns][1] += prob * rw   # weighted reward
    T[(state, action)] = [
        (p, ns, r_sum / p) for ns, (p, r_sum) in merged.items()
    ]
```

```
    return T


print("Enumerating states...")
all_states = enumerate_states()
terminal_states = [s for s in all_states if is_terminal(s)]
non_terminal    = [s for s in all_states if not is_terminal(s)]

print(f"Total states (non-terminal): {len(non_terminal)}")
print(f"Terminal states            : {len(terminal_states)}")
print(f"Total                      : {len(all_states)}")
print("\nBuilding transition table (this may take a few seconds)...")
t0 = time.time()
T = compute_transitions(all_states)
print(f"Transition table built in {time.time() - t0:.2f}s  |  entries: {len(
```

```
Enumerating states...
Total states (non-terminal): 690
Terminal states            : 230
Total                      : 920

Building transition table (this may take a few seconds)...
Transition table built in 0.02s  |  entries: 3450
```

In [10]:

```
# ──────────────────────────────────────────────────────────────────────
# Value Iteration
#
# Bellman update:
#   V(s) ← max_a  Σ_{s'} T(s,a,s') * [ R(s,a,s') + γ * V(s') ]
#
# Stopping criterion: max|V_new(s) - V_old(s)| < θ = 1e-3
# ──────────────────────────────────────────────────────────────────────

GAMMA = 0.99    # discount factor
THETA = 1e-3    # convergence threshold

def value_iteration(states, T, gamma=GAMMA, theta=THETA):
    """
    Run Value Iteration to compute the optimal value function V* and policy

    Parameters
    ----------
    states : list  — all non-terminal states
    T      : dict  — transition table from compute_transitions()
    gamma  : float — discount factor
    theta  : float — convergence threshold

    Returns
    -------
    V      : dict — optimal value function  V*(s)
    policy : dict — optimal policy          π*(s) → best action index
    iterations : int — number of sweeps until convergence
    deltas : list   — max delta per iteration (for plotting)
    """
    # Initialise V(s) = 0 for all states
    V = {s: 0.0 for s in states}
```

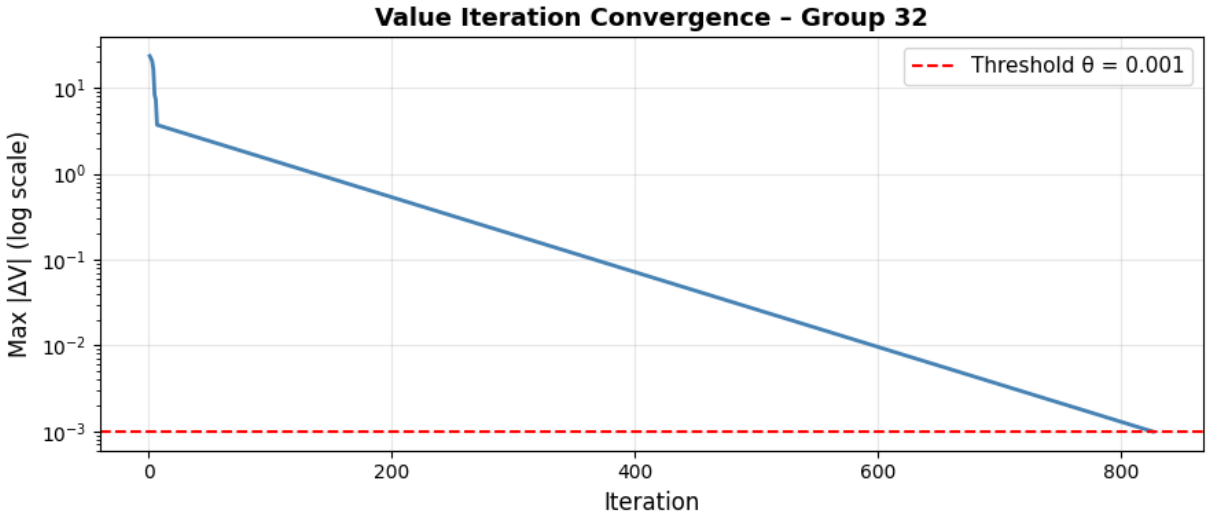
```
    iterations = 0
    deltas     = []

    while True:
        delta = 0.0
        for s in states:
            if is_terminal(s):
                continue
            old_v = V[s]
            # Compute action-values Q(s,a) for all actions
            action_values = []
            for a in [UP, DOWN, LEFT, RIGHT, HOVER]:
                if (s, a) not in T:
                    continue
                q = sum(
                    p * (r + gamma * V.get(ns, 0.0))
                    for p, ns, r in T[(s, a)]
                )
                action_values.append(q)

            if action_values:
                V[s] = max(action_values)
            delta = max(delta, abs(V[s] - old_v))

        deltas.append(delta)
        iterations += 1

        if delta < theta:
            break

    # Extract optimal policy
    policy = {}
    for s in states:
        if is_terminal(s):
            continue
        best_a, best_q = None, -np.inf
        for a in [UP, DOWN, LEFT, RIGHT, HOVER]:
            if (s, a) not in T:
                continue
            q = sum(
                p * (r + gamma * V.get(ns, 0.0))
                for p, ns, r in T[(s, a)]
            )
            if q > best_q:
                best_q = q
                best_a = a
        policy[s] = best_a

    return V, policy, iterations, deltas


print("Running Value Iteration...")
start_time = time.time()
V_star, pi_star, vi_iters, vi_deltas = value_iteration(all_states, T)
elapsed = time.time() - start_time
```

```
print(f"\n{'='*50}")
print(f"  VALUE ITERATION RESULTS")
print(f"{'='*50}")
print(f"  Convergence iterations : {vi_iters}")
print(f"  Runtime                : {elapsed:.4f} seconds")
print(f"  Final delta (error)    : {vi_deltas[-1]:.2e}")
print(f"  Threshold θ            : {THETA:.0e}")
print(f"{'='*50}")

# Sample some state values
print("\nSample V*(s) values:")
sample_states = [
    (0, 0, 10, 0b00),
    (0, 3, 8,  0b00),
    (2, 0, 6,  0b00),
    (2, 2, 5,  0b00),
    (0, 3, 8,  0b01),
    (2, 0, 6,  0b10),
]
for s in sample_states:
    a = pi_star.get(s, "terminal")
    a_name = ACTION_NAMES.get(a, str(a))
    print(f"  State {str(s):30s}  V* = {V_star.get(s, 0):8.3f}  π* = {a_name
```

```
Running Value Iteration...

==================================================
  VALUE ITERATION RESULTS
==================================================
  Convergence iterations : 826
  Runtime                : 4.9018 seconds
  Final delta (error)    : 9.91e-04
  Threshold θ            : 1e-03
==================================================

Sample V*(s) values:
  State (0, 0, 10, 0)                 V* =  406.812  π* = ↓ Down
  State (0, 3, 8, 0)                  V* =  398.116  π* = ↑ Up
  State (2, 0, 6, 0)                  V* =  411.933  π* = ← Left
  State (2, 2, 5, 0)                  V* =  406.814  π* = ← Left
  State (0, 3, 8, 1)                  V* =  381.936  π* = ↓ Down
  State (2, 0, 6, 2)                  V* =  395.892  π* = → Right
```

In [11]:
```
# ── Convergence plot ──────────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(9, 4))
ax.semilogy(range(1, vi_iters + 1), vi_deltas, color="steelblue", linewidth=
ax.axhline(y=THETA, color="red", linestyle="--", label=f"Threshold θ = {THET
ax.set_title("Value Iteration Convergence — Group 32", fontsize=13, fontweig
ax.set_xlabel("Iteration", fontsize=12)
ax.set_ylabel("Max |ΔV| (log scale)", fontsize=12)
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("DP_convergence.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
print("Convergence plot saved.")
```



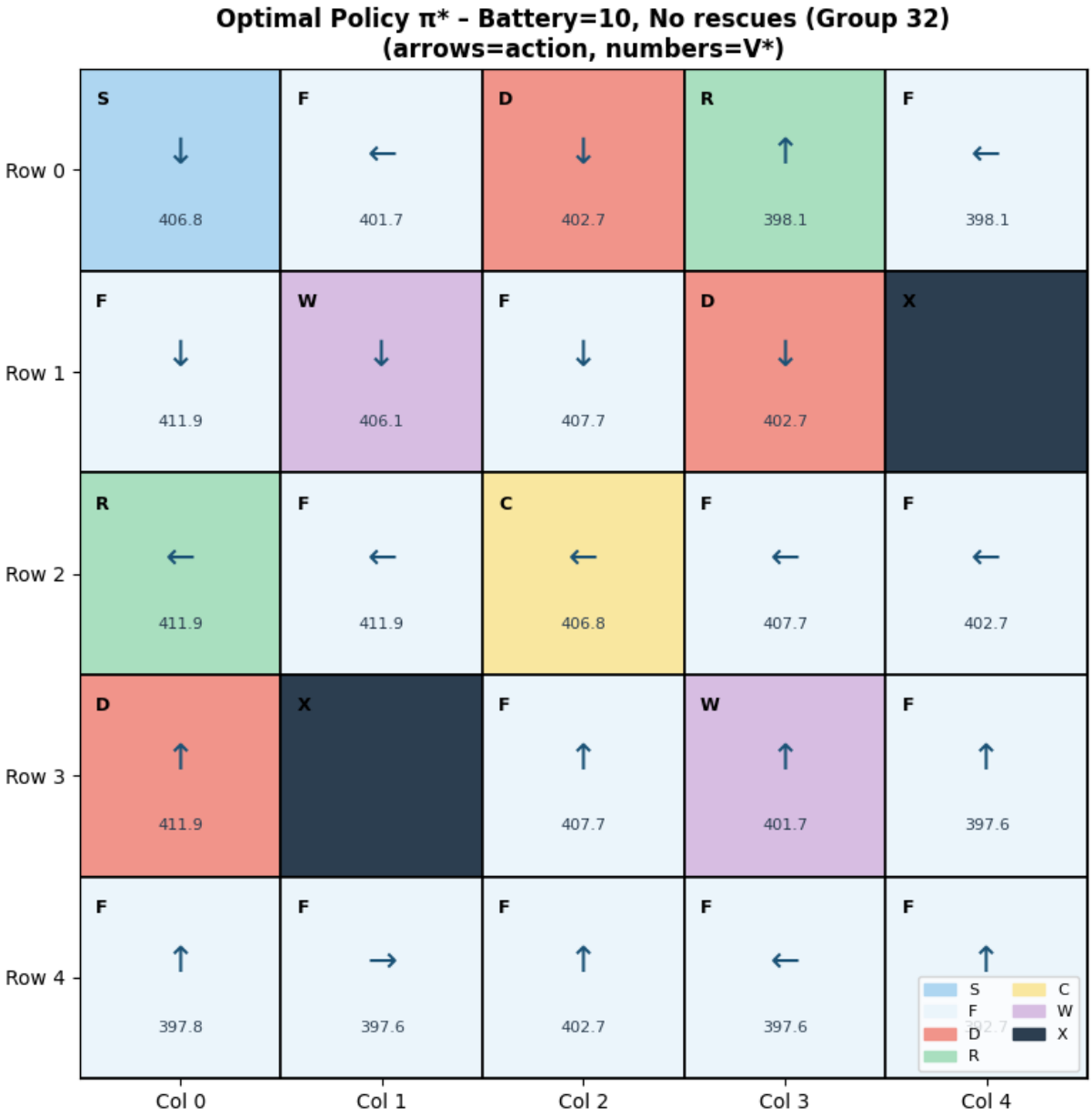

```
Convergence plot saved.
```

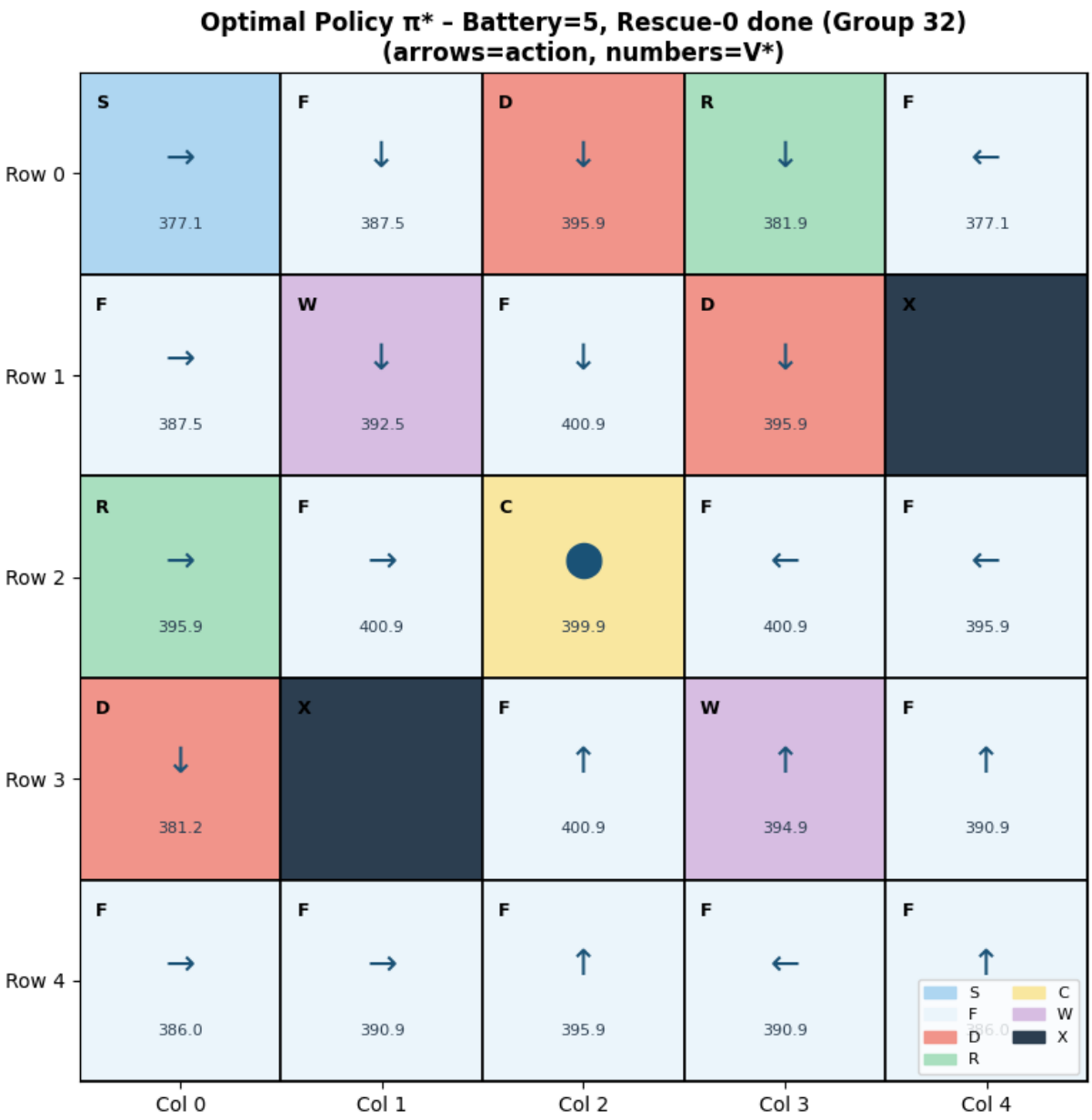
Expected Outcome 3 – Policy Visualisation (1 Mark)

Visualise the optimal policy as directional arrows on the grid, and show the value heatmap.

In [13]:
```
# ──────────────────────────────────────────────────────────────
# Policy Visualisation
# Fix battery=5 and rescue_mask=0 (no rescues done yet) to get a 5×5 slice.
# ──────────────────────────────────────────────────────────────

ACTION_ARROW = {UP: "↑", DOWN: "↓", LEFT: "←", RIGHT: "→", HOVER: "●"}
CELL_COLORS  = {
    "S": "#AED6F1", "F": "#EBF5FB", "D": "#F1948A",
    "R": "#A9DFBF", "C": "#F9E79F", "W": "#D7BDE2",
    "X": "#2C3E50"
}

def plot_policy_grid(V_star, pi_star, battery, rescue_mask, title_suffix=""):
    """
    Plot the 5×5 policy grid showing:
      - Background colour by cell type
      - Optimal action arrow for each non-terminal, non-blocked cell
      - State-value number
    """
    fig, ax = plt.subplots(figsize=(8, 8))
    ax.set_xlim(0, GRID_COLS)
    ax.set_ylim(0, GRID_ROWS)
    ax.set_aspect("equal")
    ax.invert_yaxis()

    for r in range(GRID_ROWS):
        for c in range(GRID_COLS):
            state = (r, c, battery, rescue_mask)
```

```
            base  = GRID_MAP[r][c]

            # Choose background colour
            color = CELL_COLORS.get(base, "#FFFFFF")
            rect  = mpatches.Rectangle((c, r), 1, 1,
                                       facecolor=color, edgecolor="black",
            ax.add_patch(rect)

            # Cell type label
            ax.text(c + 0.08, r + 0.18, base, fontsize=9,
                    color="black", fontweight="bold")

            if base == "X":
                continue

            # State value
            v_val = V_star.get(state, 0.0)
            ax.text(c + 0.5, r + 0.75, f"{v_val:.1f}",
                    ha="center", va="center", fontsize=8, color="#2C3E50")

            # Action arrow
            if state in pi_star:
                a     = pi_star[state]
                arrow = ACTION_ARROW.get(a, "?")
                ax.text(c + 0.5, r + 0.42, arrow,
                        ha="center", va="center", fontsize=18, color="#1A527

    ax.set_xticks(np.arange(0.5, GRID_COLS, 1))
    ax.set_xticklabels([f"Col {i}" for i in range(GRID_COLS)])
    ax.set_yticks(np.arange(0.5, GRID_ROWS, 1))
    ax.set_yticklabels([f"Row {i}" for i in range(GRID_ROWS)])
    ax.set_title(f"Optimal Policy π* — {title_suffix}\n(arrows=action, numbe
                 fontsize=12, fontweight="bold")

    # Legend
    legend_patches = [mpatches.Patch(color=v, label=k) for k, v in CELL_COLO
    ax.legend(handles=legend_patches, loc="lower right", fontsize=8, ncol=2)

    plt.tight_layout()
    return fig


# Plot policy for battery=10 (full) and no rescues yet
fig1 = plot_policy_grid(V_star, pi_star, battery=10, rescue_mask=0b00,
                        title_suffix="Battery=10, No rescues (Group 32)")
plt.savefig("DP_policy_bat10_mask00.png", dpi=150, bbox_inches="tight")
plt.show()

# Plot policy for battery=5 and one rescue done
fig2 = plot_policy_grid(V_star, pi_star, battery=5, rescue_mask=0b01,
                        title_suffix="Battery=5, Rescue-0 done (Group 32)")
plt.savefig("DP_policy_bat5_mask01.png", dpi=150, bbox_inches="tight")
plt.show()

print("Policy visualisation plots saved.")
```


Optimal Policy π* – Battery=10, No rescues (Group 32)
(arrows=action, numbers=V*)
Row 0
Row 1
Row 2
Row 3
Row 4
Col 0
Col 1
Col 2
Col 3
Col 4
S
F
D
R
F
406.8
401.7
402.7
398.1
398.1
F
W
F
D
X
411.9
406.1
407.7
402.7
R
F
C
F
F
411.9
411.9
406.8
407.7
402.7
D
X
F
W
F
411.9
407.7
401.7
397.6
F
F
F
F
F
397.8
397.6
402.7
397.6
S
F
D
R
C
W
X



Policy visualisation plots saved.

In [14]:

```
# ──────────────────────────────────────────────────────────────
# Policy Rollout: follow π* from the initial state and print trajectory
# ──────────────────────────────────────────────────────────────

def rollout(env, policy, max_steps=100):
    """
    Execute one episode following the computed optimal policy.

    Parameters
    ----------
    env    : DroneRescueEnv
    policy : dict — optimal policy π*(s) → action

    Returns
    -------
    trajectory : list of (state, action, reward) tuples
    total_reward : float
```

```
    """
    state = env.reset()
    trajectory   = []
    total_reward = 0.0

    for step in range(max_steps):
        if is_terminal(state) or env.done:
            break
        action = policy.get(state, HOVER)   # default to hover if state not
        next_state, reward, done, info = env.step(action)
        trajectory.append((state, ACTION_NAMES[action], reward, info))
        total_reward += reward
        state = next_state

    return trajectory, total_reward


np.random.seed(G)
env = DroneRescueEnv()
traj, total_r = rollout(env, pi_star)

print("Optimal Policy Rollout Trajectory")
print("=" * 70)
print(f"{'Step':>4} | {'State (r,c,bat,mask)':>26} | {'Action':>10} | {'Rewa
print("-" * 70)
for i, (s, a, r, info) in enumerate(traj):
    info_str = ", ".join(f"{k}={v}" for k, v in info.items()) if info else "
    print(f"{i+1:>4} | {str(s):>26} | {a:>10} | {r:>+7.1f} | {info_str}")
print("-" * 70)
print(f"Total episode reward: {total_r:.2f}")
print(f"Rescues completed   : {bin(env.rescue_mask).count('1')} / {NUM_RESCU
print(f"Steps taken         : {env.steps}")
env.render()
```

```
Optimal Policy Rollout Trajectory
================================================================================
Step |       State (r,c,bat,mask) |     Action |  Reward | Info
--------------------------------------------------------------------------------
   1 |               (0, 0, 10, 0) |     ↓ Down |    -1.0 |
   2 |               (1, 0, 9, 0) |     ↓ Down |   +20.0 | rescued=1
   3 |               (2, 0, 8, 2) |    → Right |    -1.0 |
   4 |               (2, 1, 7, 2) |    → Right |    +5.0 |
   5 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
   6 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
   7 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
   8 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
   9 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  10 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  11 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  12 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  13 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  14 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  15 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  16 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  17 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  18 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  19 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  20 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  21 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  22 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  23 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  24 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  25 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  26 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  27 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  28 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  29 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  30 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  31 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  32 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  33 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  34 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  35 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  36 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  37 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  38 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  39 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  40 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  41 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  42 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  43 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  44 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  45 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  46 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  47 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  48 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  49 |              (2, 2, 10, 2) |    ● Hover |    +4.0 |
  50 |              (2, 2, 10, 2) |    ● Hover |    +4.0 | termination=max_s
teps_exceeded
--------------------------------------------------------------------------------
```

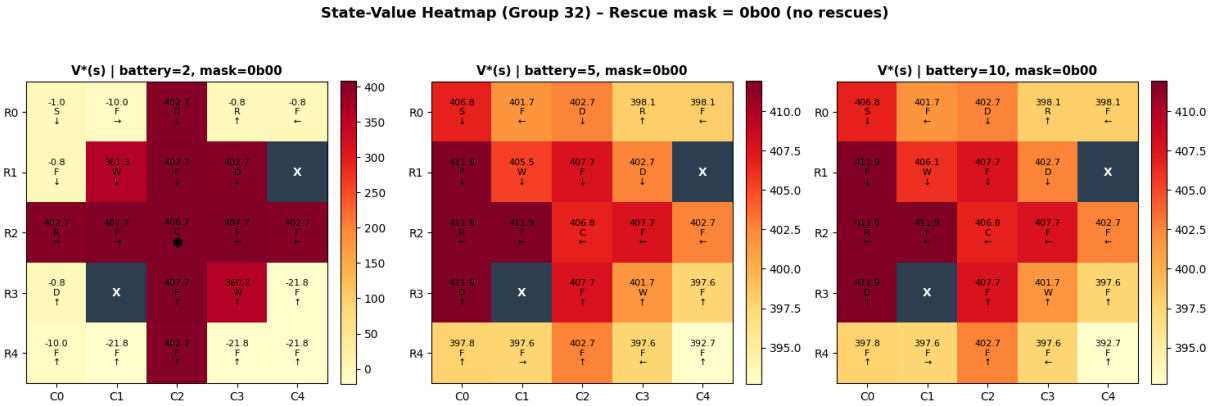
```
Total episode reward: 207.00
Rescues completed   : 1 / 2
Steps taken         : 50

Battery: 10/10 | Rescues: 1/2 | Steps: 50/50
     C0  C1  C2  C3  C4
R0 [ S   F   D   R   F ]
R1 [ F   W   F   D   X ]
R2 [ F   F   @   F   F ]
R3 [ D   X   F   W   F ]
R4 [ F   F   F   F   F ]
```

Expected Outcome 4 – State-Value Analysis (1 Mark)

Fix rescue_mask=0 (no rescues done) and vary battery level (2, 5, 10). Plot a heatmap of V*(row, col) for each battery level.

In [16]:
```
# ──────────────────────────────────────────────────────────────────────
# State-Value Heatmap
# Slice: rescue_mask = 0b00 (no rescues), vary battery ∈ {2, 5, 10}
# ──────────────────────────────────────────────────────────────────────

battery_levels = [2, 5, 10]
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for ax, bat in zip(axes, battery_levels):
    heatmap = np.full((GRID_ROWS, GRID_COLS), np.nan)
    for r in range(GRID_ROWS):
        for c in range(GRID_COLS):
            if GRID_MAP[r][c] == "X":
                heatmap[r, c] = np.nan
                continue
            state = (r, c, bat, 0b00)
            heatmap[r, c] = V_star.get(state, 0.0)

    # Mask blocked cells
    masked = np.ma.array(heatmap, mask=np.isnan(heatmap))
    cmap = plt.cm.YlOrRd.copy()
    cmap.set_bad(color="#2C3E50")  # blocked cells in dark
    im = ax.imshow(masked, cmap=cmap, vmin=np.nanmin(heatmap), vmax=np.nanma

    # Annotate each cell
    for r in range(GRID_ROWS):
        for c in range(GRID_COLS):
            ct = GRID_MAP[r][c]
            if ct == "X":
                ax.text(c, r, "X", ha="center", va="center",
                        color="white", fontsize=10, fontweight="bold")
                continue
            state = (r, c, bat, 0b00)
            v = V_star.get(state, 0.0)
            arrow = ACTION_ARROW.get(pi_star.get(state, None), "")
            ax.text(c, r, f"{v:.1f}\n{ct}\n{arrow}",
```

```
                    ha="center", va="center", fontsize=8, color="black")

    plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    ax.set_title(f"V*(s) | battery={bat}, mask=0b00", fontsize=11, fontweigh
    ax.set_xticks(range(GRID_COLS))
    ax.set_yticks(range(GRID_ROWS))
    ax.set_xticklabels([f"C{i}" for i in range(GRID_COLS)])
    ax.set_yticklabels([f"R{i}" for i in range(GRID_ROWS)])

plt.suptitle("State-Value Heatmap (Group 32) – Rescue mask = 0b00 (no rescue
             fontsize=13, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("DP_value_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()
print("Heatmap saved.")
```




```
Heatmap saved.
```

Observed Patterns in the Heatmap

1. **Rescue cells (R) have high value** — states near unrescued targets have higher V* because rescuing yields +20 reward.
2. **Charging station (C) is valuable at low battery** — at battery=2, cells adjacent to the charging station at (2,2) are assigned much higher values than at battery=10, reflecting the urgency to recharge.
3. **Danger zones (D) reduce nearby values** — states near D cells have lower V* because entering them costs −10, which discounts future rewards.
4. **High battery = higher overall values** — the heatmap becomes uniformly brighter at battery=10 versus battery=2, because more reachable paths remain before termination.
5. **Policy directs drone away from danger** — arrows around D cells consistently point the drone toward R or C cells rather than into the danger zone.

Expected Outcome 5 – DP Scalability Discussion (1 Mark)

Curse of Dimensionality

Current state space for our 5×5 environment:

- Positions: 25 cells (minus blocked) ≈ 23
- Battery levels: 11 (0–10)
- Rescue mask: 4 (2 rescue targets → 2^2 combinations)
- **Total ≈ 23 × 11 × 4 = 1,012 states**

How the state space explodes

Change	New state count	Growth factor
10×10 grid	~100 × 11 × 4 = 4,400	~4×
10×10 + battery=15	~100 × 16 × 4 = 6,400	~6×
10×10 + 5 rescue targets	~100 × 11 × 32 = 35,200	~35×
10×10 + 5 rescues + 4 weather states	~100 × 11 × 32 × 4 = 140,800	~140×

Adding k rescue targets alone multiplies states by $\mathbf{2^k}$ — an exponential explosion. With dynamic weather (e.g., wind direction changing over time), yet another dimension is added.

Why DP Becomes Impractical

1. **Memory**: Storing V(s) and T(s,a,s') for millions of states requires gigabytes of RAM.
2. **Time**: Each Value Iteration sweep iterates over all states; O(|S|²|A|) per iteration.
3. **Transition enumeration**: Building T explicitly is only feasible for small, discrete state spaces.
4. **Dynamic environments**: If the grid or wind pattern changes, the entire DP computation must restart.

How Deep RL Helps

DP limitation	Deep RL solution
Explicit state enumeration	Neural network function approximator generalises across states without enumerating all
Tabular V(s) storage	V(s) or Q(s,a) represented as NN weights (compact)

DP limitation	Deep RL solution
Re-solve on environment change	Online learning (DQN, PPO) adapts via interaction
Stochastic high-dim environments	Model-free methods (A3C, SAC) learn without building T

Methods like **Deep Q-Networks (DQN)** or **Proximal Policy Optimisation (PPO)** can handle 10×10+ grids with dynamic weather and many rescue targets by approximating the value function with a convolutional neural network that naturally captures spatial structure.

Relation to Real-World Autonomous Drones

Real drone deployments operate in continuous state spaces (GPS coordinates, altitude, velocity, battery voltage), handle sensor noise, dynamic obstacle avoidance, and multi-agent coordination — all far beyond what tabular DP can address. Industry solutions (e.g., Amazon Prime Air, Zipline medical delivery) use combinations of:

- **Deep RL** for policy learning in simulation
- **Model Predictive Control** for real-time replanning
- **Transfer learning** to bridge the sim-to-real gap

DP remains valuable as a **theoretical baseline** and for **small, well-defined sub-problems** (e.g., battery management under a fixed route), but the full autonomous rescue problem demands Deep RL.